

2026-05-05-p1-f14-sandboxed-shell-execution

P1-F14 — Sandboxed Shell Execution Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Goal: Ship a real, kernel-enforced Linux sandbox for shell commands invoked by the agent. New `shell_sandboxed` tool wraps every command in either bubblewrap (preferred when on PATH) or a native Go `Cloneflags`-based namespace fallback. Default-DENY network. CONST-033 power-management deny-list rejected BEFORE any subprocess spawns. Fail-closed on hosts without bubblewrap AND without unprivileged user namespaces — never a silent unsandboxed run. User surfaces: tool `shell_sandboxed`, slash command `/sandbox` (`status/test/policy`), and config file `~/ .config/helixcode/sandbox.yaml` (mode 0600).

Architecture: New `internal/tools/sandbox/` package with `types.go`, `detector.go`, `bubblewrap_backend.go`, `native_backend.go` (+ `native_backend_other.go` for non-Linux), `manager.go`, `sandboxed_shell_tool.go`, `config_loader.go`, plus a small re-exec helper at `internal/tools/sandbox/cmd/native_helper/main.go`. Slash command at `internal/commands/sandbox_command.go`. `internal/tools/registry.go` gets a single new method `SetSandboxManager(*sandbox.SandboxManager)` (mirrors F13's `SetLSPManager`) which lazily registers the tool. `cmd/cli/main.go` runs `sandbox.Detect()` once, loads the config, builds the manager, wires it into the registry and slash registry. **The existing `internal/tools/shell/` package is left untouched** (pre-existing build failures predate F09 and are out of scope per Q4=B).

Tech Stack: Go 1.26, testify v1.11, spf13/cobra v1.8 — all already present. **NO new external deps:** bubblewrap is invoked as a subprocess via `os/exec`; native namespace setup uses `syscall.SysProcAttr.Cloneflags` from the standard library; YAML loading uses the already-present `gopkg.in/yaml.v3 v3.0.1`. Confirmed: `go.mod` already carries `gopkg.in/yaml.v3 v3.0.1` and `github.com/fsnotify/fsnotify v1.9.0`; nothing new needed.

Spec: `docs/superpowers/specs/2026-05-05-p1-f14-sandboxed-shell-execution-design.md` (commit 067e5d9)

Working directory for go commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F14 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" \
    internal/tools/sandbox internal/commands/sandbox_command.go &&
    echo BLUFF || echo clean
```

Must always print `clean`.

Anti-bluff hot zone: §5.2 of the spec — bubblewrap and unprivileged user namespaces may BOTH be missing on the test machine. The Challenge MUST present THREE clearly separated sections: DETECTOR + FAIL-CLOSED (always runs), BUBBLEWRAP (gated), NATIVE (gated). Phase A is non-negotiable: it asserts the fail-closed error message verbatim, AND it asserts the CONST-033 deny-list rejects `systemctl suspend` BEFORE any backend is invoked (counter on a fake backend stays at 0). Real-backend tests skip with SKIP-OK: P1-F14 <missing capability> (install: <hint>). Bare skips break make no-silent-skips.

Why this is the most consequential anti-bluff in the programme: an unsandboxed run that claims to be sandboxed is a security failure with the agent's full FS + network reach. Phase A's fail-closed assertion (against a forced-empty `SandboxCapabilities`) is the single most important test in F14.

Task list



P1-F14-T01 — bootstrap evidence + advance PROGRESS to F14



P1-F14-T02 — `internal/tools/sandbox/types.go`: `SandboxConfig` + `SandboxPolicy` + `SandboxCapabilities` + `SandboxRequest` + `SandboxResult` + `SandboxBackend` interface + `ConstitutionalDenyList` constants (TDD)



P1-F14-T03 — `internal/tools/sandbox/detector.go`: `Detect()` probes bwrap + users + cgroups v2 + GOOS via injected `Probe` interface (TDD)



P1-F14-T04 — `internal/tools/sandbox/bubblewrap_backend.go`: `argv` builder + `Run` via `os/exec.CommandContext` (TDD with synthetic `argv` assertions on a fake `SubprocessRunner`)



P1-F14-T05 — `internal/tools/sandbox/native_backend.go` (+ `native_backend_other.go`): `SysProcAttr.Cloneflags` setup + small re-exec helper (TDD; gated tests when users enabled)



P1-F14-T06 — `internal/tools/sandbox/manager.go`: backend selection + CONST-033 deny-list + user deny-list + network gating + timeout (TDD with fake backend + spawn-counter)



P1-F14-T07 — `internal/tools/sandbox/sandboxed_shell_tool.go`: `Tool` interface impl, registered name `shell_sandboxed` (TDD)



P1-F14-T08 — `internal/tools/sandbox/config_loader.go`: YAML loader for `~/config/helixcode/sandbox.yaml` + secret-safe writer (mode 0600, parent 0700; mirrors F12 `wizard_writer`) (TDD)



P1-F14-T09 — `/sandbox` slash command at `internal/commands/sandbox_command.go` (status/test/policy) (TDD)



P1-F14-T10 — `main.go` wiring (`Detector` + `Manager` + tool registration + slash registration) + integration tests (gated, SKIP-OK on missing bwrap/users)



P1-F14-T11 — Challenge harness (Phase A always-runs detector + fail-closed; Phase B bwrap-gated; Phase C native-gated)



P1-F14-T12 — Feature 14 close-out + push 4 remotes non-force

Task 1: Bootstrap

Append F14 evidence section header (spec 067e5d9), update PROGRESS current focus to F14, insert F14 task list (12 items) after F13's. Commit docs (P1-F14-T01): bootstrap Phase 1 / Feature 14 evidence + advance PROGRESS.

Task 2: types.go (TDD)

Files: new helix_code/internal/tools/sandbox/types.go, new helix_code/internal/tools/sandbox/types_test.go.

Define SandboxBackendKind enum (None/Bubblewrap/Native) with String(), SandboxCapabilities, SandboxPolicy, SandboxRequest, SandboxResult, SandboxBackend interface, SandboxConfig, and the ConstitutionalDenyList slice from spec §3.3. Add DefaultSandboxConfig(), DefaultSandboxPolicy(), and MergedDenyList(cfg SandboxConfig) []string (CONST-033 + user, deduped, marker-prefixed for [CONST-033] rendering).

Failing test FIRST:

```
func TestConstitutionalDenyList_ContainsAllPowerStateCommands(t
    *testing.T) {
    must := []string{"systemctl suspend", "systemctl hibernate",
        "systemctl poweroff",
        "systemctl reboot", "shutdown", "halt", "poweroff",
        "reboot", "kexec",
        "pm-suspend", "loginctl suspend", "loginctl poweroff"}
    for _, m := range must {
        require.Contains(t, ConstitutionalDenyList, m,
            "CONST-033 missing %q", m)
    }
}
```

```
func TestMergedDenyList_DedupsAndMarksConstitutional(t
    *testing.T) {
    cfg := SandboxConfig{DenyList: []string{"rm -rf /",
        "shutdown"}} // shutdown overlaps CONST-033
    merged := MergedDenyList(cfg)
    seen := map[string]int{}
    for _, e := range merged { seen[stripMarker(e)]++ }
    require.Equal(t, 1, seen["shutdown"], "shutdown must appear
        once even though both lists contain it")
}
```

```

    require.Equal(t, 1, seen["rm -rf /"])
}

func TestDefaultSandboxPolicy_NetworkDeniedByDefault(t
    *testing.T) {
    p := DefaultSandboxPolicy()
    require.False(t, p.AllowNetwork, "Q3=A: network MUST default
        to deny")
    require.Equal(t, 30*time.Second, p.Timeout)
}

```

Subject: feat(P1-F14-T02): sandbox types + ConstitutionalDenyList + default-DENY policy.

Task 3: detector.go (TDD)

Files: new helix_code/internal/tools/sandbox/detector.go, new helix_code/internal/tools/sandbox/detector_test.go.

Detect() SandboxCapabilities calls a small Probe interface so unit tests inject canned responses:

```

type Probe interface {
    LookPath(name string) (string, error)
    ReadSysctl(path string) (string, error) // returns trimmed
        contents of /proc/sys/...
    StatExists(path string) bool
    GOOS() string
    Getuid() int
}

func Detect() SandboxCapabilities //
    uses defaultProbe (real syscalls)
func DetectWith(p Probe) SandboxCapabilities //
    injectable for tests
func SelectBackend(caps SandboxCapabilities, runner
    SubprocessRunner) (SandboxBackend, error)

```

Test:

```

func TestDetect_BubblewrapPresent_FillsPath(t *testing.T) {
    p := fakeProbe{lookPath: map[string]string{"bwrap": "/usr/
        bin/bwrap"}, gooss: "linux", uid: 1000}
    caps := DetectWith(p)
    require.Equal(t, "/usr/bin/bwrap", caps.BubblewrapPath)
    require.Equal(t, "linux", caps.GOOS)
}

func TestDetect_UnprivilegedUserns_FromSysctl(t *testing.T) {

```

```

    p := fakeProbe{sysctl: map[string]string{"/proc/sys/kernel/
        unprivileged_usersns_clone": "1"}, gooss: "linux"}
    require.True(t, DetectWith(p).UnprivilegedUsersns)
    p.sysctl["/proc/sys/kernel/unprivileged_usersns_clone"] = "0"
    require.False(t, DetectWith(p).UnprivilegedUsersns)
}

func TestDetect_CgroupsV2_FromControllersFile(t *testing.T) {
    p := fakeProbe{stat: map[string]bool{"/sys/fs/cgroup/
        cgroup.controllers": true}, gooss: "linux"}
    require.True(t, DetectWith(p).CgroupsV2Mounted)
}

func TestSelectBackend_PrefersBubblewrap(t *testing.T) {
    caps := SandboxCapabilities{GOOS: "linux", BubblewrapPath: "/
        usr/bin/bwrap", UnprivilegedUsersns: true}
    b, err := SelectBackend(caps, fakeRunner{})
    require.NoError(t, err)
    require.Equal(t, BackendBubblewrap, b.Kind())
}

func TestSelectBackend_FallsBackToNative(t *testing.T) {
    caps := SandboxCapabilities{GOOS: "linux", BubblewrapPath:
        "", UnprivilegedUsersns: true}
    b, err := SelectBackend(caps, fakeRunner{})
    require.NoError(t, err)
    require.Equal(t, BackendNative, b.Kind())
}

func TestSelectBackend_FailsClosed_NoBwrap_NoUsersns(t
    *testing.T) {
    _, err := SelectBackend(SandboxCapabilities{GOOS: "linux"},
        fakeRunner{})
    require.ErrorIs(t, err, ErrSandboxUnavailable)
    require.Contains(t, err.Error(), "install bubblewrap")
    require.Contains(t, err.Error(), "unprivileged_usersns_clone")
}

func TestSelectBackend_FailsClosed_NonLinux(t *testing.T) {
    _, err := SelectBackend(SandboxCapabilities{GOOS: "darwin",
        BubblewrapPath: "/opt/homebrew/bin/bwrap"}, fakeRunner{})
    require.ErrorIs(t, err, ErrSandboxUnavailable)
    require.Contains(t, err.Error(), "Linux only")
}

```

Subject: feat(P1-F14-T03): SandboxDetector + SelectBackend with explicit fail-closed semantics.

Task 4: bubblewrap_backend.go (TDD)

Files: new helix_code/internal/tools/sandbox/bubblewrap_backend.go,
new helix_code/internal/tools/sandbox/bubblewrap_backend_test.go.

BubblewrapBackend builds argv from SandboxRequest and runs via injected SubprocessRunner (interface boundary so unit tests don't spawn real subprocesses).

```
type SubprocessRunner interface {
    Run(ctx context.Context, name string, args []string,
        stdin io.Reader, env []string, cwd string,
        maxBytes int64) (stdout, stderr []byte, exit int,
        truncated bool, err error)
}

type BubblewrapBackend struct { Path string; Runner
    SubprocessRunner }

func (b *BubblewrapBackend) Kind() SandboxBackendKind { return
    BackendBubblewrap }
func (b *BubblewrapBackend) BuildArgv(req *SandboxRequest)
    []string // pure, deterministic
func (b *BubblewrapBackend) Run(ctx context.Context, req
    *SandboxRequest) (*SandboxResult, error)
```

Argv shape (per spec §4.3) is asserted by tests against the pure BuildArgv. Run delegates to Runner.Run(ctx, b.Path, argv, ...).

Tests:

```
func TestBubblewrapBackend_ArgvShape_DefaultDeniesNetwork(t
    *testing.T) {
    b := &BubblewrapBackend{Path: "/usr/bin/bwrap"}
    req := &SandboxRequest{Command: "echo hi", Policy:
        DefaultSandboxPolicy()}
    argv := b.BuildArgv(req)
    require.Contains(t, argv, "--unshare-net")
    require.Contains(t, argv, "--unshare-pid")
    require.Contains(t, argv, "--unshare-user") // users
    require.Contains(t, argv, "--die-with-parent")
}

func TestBubblewrapBackend_ArgvShape_NetworkAllowedDropsUnshareNet(t
    *testing.T) {
    b := &BubblewrapBackend{Path: "/usr/bin/bwrap"}
    pol := DefaultSandboxPolicy(); pol.AllowNetwork = true
    argv := b.BuildArgv(&SandboxRequest{Command: "curl x",
        Policy: pol})
    require.NotContains(t, argv, "--unshare-net")
}
```

```

func TestBubblewrapBackend_ArgvShape_RootIsReadOnly_CwdIsReadWrite(t
    *testing.T) {
    pol := DefaultSandboxPolicy(); pol.Cwd = "/tmp/work"
    argv := (&BubblewrapBackend{Path: "/usr/bin/
        bwrap"}).BuildArgv(&SandboxRequest{Command: "x", Policy:
            pol})
    requireSliceContainsPair(t, argv, "--ro-bind", "/")
    requireSliceContainsPair(t, argv, "--bind", "/tmp/work")
    requireSliceContainsPair(t, argv, "--chdir", "/tmp/work")
}

func TestBubblewrapBackend_Run_DelegatesToRunnerWithBwrapPath(t
    *testing.T) {
    runner := &recordingRunner{stdout: []byte("hi\n"), exit: 0}
    b := &BubblewrapBackend{Path: "/usr/bin/bwrap", Runner:
        runner}
    res, err := b.Run(context.Background(),
        &SandboxRequest{Command: "echo hi", Policy:
            DefaultSandboxPolicy()})
    require.NoError(t, err)
    require.Equal(t, "/usr/bin/bwrap", runner.lastName)
    require.Equal(t, 0, res.ExitCode)
    require.Equal(t, []byte("hi\n"), res.Stdout)
}

```

Subject: feat(P1-F14-T04): BubblewrapBackend with deterministic argv builder.

Task 5: native_backend.go (TDD; gated tests)

Files: new helix_code/internal/tools/sandbox/native_backend.go (//go:build linux), new helix_code/internal/tools/sandbox/native_backend_other.go (//go:build !linux), new helix_code/internal/tools/sandbox/native_backend_test.go (//go:build linux), new helix_code/internal/tools/sandbox/cmd/native_helper/main.go (the re-exec helper).

NativeBackend sets cmd.SysProcAttr.Cloneflags to CLONE_NEWPID|CLONE_NEWNS|CLONE_NEWUTS|CLONE_NEWIPC|CLONE_NEWUSER, plus CLONE_NEWNET when network denied. UidMappings/GidMappings map container UID 0 to the host's current UID/GID. The re-exec helper at internal/tools/sandbox/cmd/native_helper/main.go runs with PID 1 in the new namespaces, remounts /proc, then execves /bin/sh -c <command>. Helper path resolved at startup via sibling lookup near os.Executable(); if absent, native backend's Run returns ErrNativeHelperMissing.

Non-Linux stub:

```
//go:build !linux
package sandbox

type NativeBackend struct{}
func (b *NativeBackend) Kind() SandboxBackendKind { return
    BackendNative }
func (b *NativeBackend) Run(ctx context.Context, req
    *SandboxRequest) (*SandboxResult, error) {
    return nil, fmt.Errorf("%w: native backend requires Linux",
        ErrSandboxUnavailable)
}
```

Tests (Linux only, gated on usersn):

```
//go:build linux
func skipIfNoUsersn(t *testing.T) {
    b, _ := os.ReadFile("/proc/sys/kernel/
        unprivileged_usersn_clone")
    if strings.TrimSpace(string(b)) != "1" {
        t.Skip("SKIP-OK: P1-F14 unprivileged user namespaces
            disabled (echo 1 > /proc/sys/kernel/
                unprivileged_usersn_clone)")
    }
}

func TestNativeBackend_Cloneflags_IncludeUsersn(t *testing.T) {
    skipIfNoUsersn(t)
    cf := nativeCloneflags(DefaultSandboxPolicy())
    require.NotZero(t, cf & syscall.CLONE_NEWUSER)
    require.NotZero(t, cf & syscall.CLONE_NEWPID)
    require.NotZero(t, cf & syscall.CLONE_NEWNS)
}

func TestNativeBackend_Cloneflags_AddsNewnetWhenNetDenied(t
    *testing.T) {
    skipIfNoUsersn(t)
    pol := DefaultSandboxPolicy() // AllowNetwork=false
    require.NotZero(t, nativeCloneflags(pol) &
        syscall.CLONE_NEWNET)
    pol.AllowNetwork = true
    require.Zero(t, nativeCloneflags(pol) & syscall.CLONE_NEWNET)
}

func TestNativeBackend_RunsEcho(t *testing.T) {
    skipIfNoUsersn(t)
    helper := buildNativeHelper(t)
    b := &NativeBackend{HelperPath: helper}
```

```

    res, err := b.Run(context.Background(),
        &SandboxRequest{Command: "echo p1f14-native-ok", Policy:
            DefaultSandboxPolicy()})
    require.NoError(t, err)
    require.Equal(t, 0, res.ExitCode)
    require.Contains(t, string(res.Stdout), "p1f14-native-ok")
}

```

Subject: feat(P1-F14-T05): NativeBackend with Cloneflags userns + native_helper re-exec.

Task 6: manager.go (TDD with fake backend + spawn-counter)

Files: new helix_code/internal/tools/sandbox/manager.go, new helix_code/internal/tools/sandbox/manager_test.go.

SandboxManager.Execute (per spec §4.2): backend-nil check → CONST-033 deny-list → user deny-list → ctx timeout → Backend.Run. Both deny-list checks use tokenised argv-head match AND word-boundary regex on the raw command (so `systemctl suspend; ls` and `bash -c 'systemctl suspend'` are both caught).

Tests use a countingBackend so we can assert the deny-list rejected the command BEFORE spawn:

```

type countingBackend struct { runs int; result *SandboxResult }
func (c *countingBackend) Kind() SandboxBackendKind { return
    BackendBubblewrap }
func (c *countingBackend) Run(ctx context.Context, req
    *SandboxRequest) (*SandboxResult, error) {
    c.runs++; return c.result, nil
}

```

```

func TestManager_FailsClosed_WhenBackendNil(t *testing.T) {
    m := NewSandboxManager(SandboxManagerOptions{Capabilities:
        SandboxCapabilities{G00S: "darwin"}, Config:
        DefaultSandboxConfig(), Backend: nil})
    _, err := m.Execute(context.Background(),
        &SandboxRequest{Command: "echo hi", Policy:
        DefaultSandboxPolicy()})
    require.ErrorIs(t, err, ErrSandboxUnavailable)
}

```

```

func TestManager_RejectsConstitutionalDenyList_BeforeSpawn(t
    *testing.T) {
    cb := &countingBackend{result: &SandboxResult{ExitCode: 0}}
    m := NewSandboxManager(SandboxManagerOptions{Backend: cb,
        Config: DefaultSandboxConfig()})
}

```

```

for _, cmd := range []string{"systemctl suspend",
    "systemctl suspend", "shutdown -h now",
    "bash -c 'systemctl suspend'", "ls && systemctl reboot"}
{
    _, err := m.Execute(context.Background(),
        &SandboxRequest{Command: cmd, Policy:
            DefaultSandboxPolicy()})
    require.ErrorIs(t, err, ErrConstitutionalDeny, "cmd %q
        must be CONST-033 rejected", cmd)
}
require.Equal(t, 0, cb.runs, "no subprocess must be spawned
    for CONST-033 hits")
}

func TestManager_RejectsUserDenyList(t *testing.T) {
    cb := &countingBackend{}
    cfg := DefaultSandboxConfig(); cfg.DenyList = []string{"rm
        -rf"}
    m := NewSandboxManager(SandboxManagerOptions{Backend: cb,
        Config: cfg})
    _, err := m.Execute(context.Background(),
        &SandboxRequest{Command: "rm -rf /tmp/foo", Policy:
            DefaultSandboxPolicy()})
    require.ErrorIs(t, err, ErrPolicyDeny)
    require.Equal(t, 0, cb.runs)
}

func TestManager_AppliesTimeout(t *testing.T) { /* ctx cancel
    propagates to backend */ }
func TestManager_TruncatesOutputAtMaxBytes(t *testing.T) { /*
    fake backend returns long stdout */ }

```

Subject: feat(P1-F14-T06): SandboxManager with CONST-033 deny-list + user deny-list + fail-closed gate.

Task 7: sandboxed_shell_tool.go (TDD)

Files: new helix_code/internal/tools/sandbox/sandboxed_shell_tool.go, new helix_code/internal/tools/sandbox/sandboxed_shell_tool_test.go.

SandboxedShellTool implements the Tool interface from internal/tools/registry.go. Tool name: shell_sandboxed. Schema: command (string, required), network (bool, default false), timeout_seconds (int, default 30; max 600), cwd (string, optional). Execute builds SandboxRequest from params, calls manager.Execute, returns a result map.

Note the Tool interface lives in package tools, but our tool is in package sandbox — to avoid the name collision noted in spec §3.5 we define an adapter in internal/tools/

registry.go that wraps *sandbox.SandboxedShellTool and satisfies tools.Tool. The wrapper is trivial (just forwards calls); it exists so internal/tools/sandbox does not need to import internal/tools (no circular dep).

Tests:

```
func TestSandboxedShellTool_Name_IsShellSandboxed(t *testing.T) {
    require.Equal(t, "shell_sandboxed",
        (&SandboxedShellTool{}).Name())
}

func TestSandboxedShellTool_NetworkDefaultsFalse(t *testing.T) {
    fb := &countingBackend{result: &SandboxResult{ExitCode: 0,
        Stdout: []byte("ok")}}
    m := NewSandboxManager(SandboxManagerOptions{Backend: fb,
        Config: DefaultSandboxConfig()})
    tool := NewSandboxedShellTool(m)
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{"command": "echo ok"})
    require.NoError(t, err)
    require.Equal(t, 1, fb.runs)
    require.False(t, fb.lastReq.Policy.AllowNetwork, "network
        MUST default to false")
}

func TestSandboxedShellTool_NetworkOptIn(t *testing.T) {
    fb := &countingBackend{result: &SandboxResult{ExitCode: 0}}
    m := NewSandboxManager(SandboxManagerOptions{Backend: fb,
        Config: DefaultSandboxConfig()})
    tool := NewSandboxedShellTool(m)
    _, _ = tool.Execute(context.Background(),
        map[string]interface{}{"command": "curl x", "network":
            true})
    require.True(t, fb.lastReq.Policy.AllowNetwork)
}

func TestSandboxedShellTool_ResultMapShape(t *testing.T) {
    fb := &countingBackend{result: &SandboxResult{ExitCode: 0,
        Stdout: []byte("hi"), Backend: BackendBubblewrap,
        Duration: 5 * time.Millisecond}}
    m := NewSandboxManager(SandboxManagerOptions{Backend: fb,
        Config: DefaultSandboxConfig()})
    tool := NewSandboxedShellTool(m)
    out, err := tool.Execute(context.Background(),
        map[string]interface{}{"command": "echo hi"})
    require.NoError(t, err)
    res := out.(map[string]interface{})
    require.Equal(t, 0, res["exit_code"])
    require.Equal(t, "hi", res["stdout"])
```

```
    require.Equal(t, "bubblewrap", res["backend"])
}
```

Subject: feat(P1-F14-T07): SandboxedShellTool implementing Tool interface as shell_sandboxed.

Task 8: config_loader.go (TDD; mirrors F12 wizard_writer)

Files: new helix_code/internal/tools/sandbox/config_loader.go, new helix_code/internal/tools/sandbox/config_loader_test.go.

```
func LoadConfig(path string) (SandboxConfig, error) // ""
    → ~/.config/helixcode/sandbox.yaml; missing OK → defaults
func WriteConfig(path string, cfg SandboxConfig) error //
    O_CREATE|O_WRONLY|O_EXCL, 0o600; parent 0o700
func DefaultConfigPath() string //
    XDG_CONFIG_HOME aware
```

Loader rejects files with permissions wider than 0o600 with ErrInsecurePerms. Mirrors F12 wizard_writer.go exactly.

Tests:

```
func TestLoadConfig_MissingFile_ReturnsDefaults_NoError(t
    *testing.T) {
    cfg, err := LoadConfig(filepath.Join(t.TempDir(),
        "absent.yaml"))
    require.NoError(t, err)
    require.False(t, cfg.DefaultPolicy.AllowNetwork)
}

func TestLoadConfig_ParsesYAML(t *testing.T) {
    p := filepath.Join(t.TempDir(), "s.yaml")
    require.NoError(t, os.WriteFile(p, []byte("deny_list:
        \n - rm -rf\nmax_output_bytes: 4096\n"), 0o600))
    cfg, err := LoadConfig(p)
    require.NoError(t, err)
    require.Equal(t, []string{"rm -rf"}, cfg.DenyList)
    require.Equal(t, int64(4096), cfg.MaxOutputBytes)
}

func TestLoadConfig_RejectsWorldReadable(t *testing.T) {
    p := filepath.Join(t.TempDir(), "s.yaml")
    require.NoError(t, os.WriteFile(p, []byte("{}"), 0o644))
    _, err := LoadConfig(p)
    require.ErrorIs(t, err, ErrInsecurePerms)
}
```

```

func TestWriteConfig_EnforcesMode0600_AndParent0700(t
    *testing.T) {
    dir := t.TempDir()
    p := filepath.Join(dir, "deep", "sandbox.yaml")
    require.NoError(t, WriteConfig(p, DefaultSandboxConfig()))
    info, _ := os.Stat(p); require.Equal(t,
        fs.FileMode(0o600), info.Mode().Perm())
    parent, _ := os.Stat(filepath.Dir(p)); require.Equal(t,
        fs.FileMode(0o700), parent.Mode().Perm())
}

func TestWriteConfig_FailsIfFileAlreadyExists(t *testing.T) { /*
    O_EXCL */ }

```

Subject: feat(P1-F14-T08): sandbox.yaml YAML loader + secret-safe writer (0600/0700).

Task 9: /sandbox slash command (TDD)

Files: new helix_code/internal/commands/sandbox_command.go, new helix_code/internal/commands/sandbox_command_test.go.

Mirrors F13 lsp_command.go pattern: defines a small SandboxManager interface in the commands package so the slash command is testable with a fake while still letting main.go pass the real *sandbox.SandboxManager.

Subcommands: - /sandbox status — capabilities table (GOOS, bwrap path or “not installed”, users yes/no, cgroups v2 yes/no, selected backend, selection reason). - /sandbox test — runs /bin/echo p1f14-sandbox-ok via manager.Execute with default policy; prints exit code, captured stdout (trim, truncate to 256 bytes), backend used. - /sandbox policy — prints active SandboxPolicy + the merged deny-list with [CONST-033] markers.

Tests:

```

func TestSandboxCommand_Status_RendersCapabilitiesTable(t
    *testing.T) { /* fake mgr returns canned caps */ }
func TestSandboxCommand_Test_RunsEchoAndPrintsExitCode(t
    *testing.T) { /* fake mgr returns SandboxResult */ }
func TestSandboxCommand_Policy_PrintsConstitutionalEntriesWithMarker(t
    *testing.T) {
    res := /* exec /sandbox policy with default config */
    require.Contains(t, res.Output, "[CONST-033]")
    require.Contains(t, res.Output, "systemctl suspend")
}

```

Subject: feat(P1-F14-T09): /sandbox slash command (status / test / policy).

Task 10: main.go wiring + integration test

Files: modify `helix_code/cmd/cli/main.go`, modify `helix_code/internal/tools/registry.go`, new `helix_code/tests/integration/sandbox_test.go` (`//go:build integration`).

`registry.go` modifications:

```
sandboxManager *sandbox.SandboxManager // optional; nil disables
    shell_sandboxed registration

func (r *ToolRegistry) SetSandboxManager(m
    *sandbox.SandboxManager) {
    r.mu.Lock(); defer r.mu.Unlock()
    r.sandboxManager = m
    r.tools["shell_sandboxed"] =
        newSandboxedShellToolAdapter(m) // adapter wraps
        *sandbox.SandboxedShellTool
}
```

`main.go` startup wiring (per spec §4.1):

```
caps := sandbox.Detect()
cfg, cfgErr := sandbox.LoadConfig("") // missing OK
if cfgErr != nil && !errors.Is(cfgErr, fs.ErrNotExist) {
    log.Printf("WARN: %v", cfgErr) }
runner := sandbox.NewExecRunner()
backend, selErr := sandbox.SelectBackend(caps, runner)
if selErr != nil { log.Printf("INFO: %v", selErr) }
mgr := sandbox.NewSandboxManager(sandbox.SandboxManagerOptions{
    Capabilities: caps, Config: cfg, Backend: backend,
})
defer mgr.Close(context.Background())
registry.SetSandboxManager(mgr)
slashRegistry.Register(commands.NewSandboxCommand(mgr))
```

Integration test (gated, per spec §5.2):

```
//go:build integration
// +build integration

func TestSandbox_Bubblewrap_RunsEcho(t *testing.T) {
    if _, err := exec.LookPath("bwrap"); err != nil {
        t.Skip("SKIP-OK: P1-F14 bubblewrap not on PATH (install:
            apt install bubblewrap OR dnf install bubblewrap)")
    }
    caps := sandbox.Detect()
    backend, err := sandbox.SelectBackend(caps,
        sandbox.NewExecRunner())
    require.NoError(t, err)
```

```

require.Equal(t, sandbox.BackendBubblewrap, backend.Kind())
m :=
    sandbox.NewSandboxManager(sandbox.SandboxManagerOptions{Capabilities:
        caps, Config: sandbox.DefaultSandboxConfig(), Backend: backend})
res, err := m.Execute(context.Background(),
    &sandbox.SandboxRequest{Command: "echo p1f14-bwrap-ok",
        Policy: sandbox.DefaultSandboxPolicy()})
require.NoError(t, err)
require.Equal(t, 0, res.ExitCode)
require.Contains(t, string(res.Stdout), "p1f14-bwrap-ok")
}

func TestSandbox_Bubblewrap_NetworkDeniedByDefault(t *testing.T)
    { /* curl --max-time 2 https://example.com → non-zero
        */ }
func TestSandbox_Native_RunsEcho(t *testing.T) { /* SKIP-OK on
    users=0 */ }
func TestSandbox_FailsClosedWhenNoBackend(t *testing.T) {
    // Never skipped — proves fail-closed semantics regardless of
    // host capabilities.
    m :=
        sandbox.NewSandboxManager(sandbox.SandboxManagerOptions{Capabilities:
            sandbox.SandboxCapabilities{}, Config:
            sandbox.DefaultSandboxConfig(), Backend: nil})
    _, err := m.Execute(context.Background(),
        &sandbox.SandboxRequest{Command: "echo hi", Policy:
            sandbox.DefaultSandboxPolicy()})
    require.ErrorIs(t, err, sandbox.ErrSandboxUnavailable)
    require.Contains(t, err.Error(), "install bubblewrap")
}
func TestSandbox_RejectsConstitutionalDenyList(t *testing.T) {
    // Never skipped — proves CONST-033 enforcement.
    cb := &countingBackend{result:
        &sandbox.SandboxResult{ExitCode: 0}}
    m :=
        sandbox.NewSandboxManager(sandbox.SandboxManagerOptions{Backend:
            cb, Config: sandbox.DefaultSandboxConfig()})
    _, err := m.Execute(context.Background(),
        &sandbox.SandboxRequest{Command: "systemctl suspend",
            Policy: sandbox.DefaultSandboxPolicy()})
    require.ErrorIs(t, err, sandbox.ErrConstitutionalDeny)
    require.Equal(t, 0, cb.runs)
}

```

Subject: feat(P1-F14-T10): wire SandboxManager into main.go + /
 sandbox + gated integration tests.

Task 11: Challenge with runtime evidence

Files: new helix_code/tests/integration/cmd/plf14_challenge/main.go, new challenges/p1-f14-sandboxed-shell-execution/CHALLENGE.md, new challenges/p1-f14-sandboxed-shell-execution/run.sh.

Output skeleton:

```
=== DETECTOR + FAIL-CLOSED (always runs) ===
[PASS] detector: G00S=linux, bwrap=/usr/bin/bwrap (or "not installed"), us
[PASS] select-backend: chose bubblewrap (or native, or fail-closed)
[PASS] fail-closed: forced empty SandboxCapabilities returns ErrSandboxUna
[PASS] fail-closed: error message contains "install bubblewrap" AND "unpri
[PASS] CONST-033: "systemctl suspend" rejected with ErrConstitutionalDeny
[PASS] CONST-033: nested form "bash -c 'systemctl suspend'" also rejected

=== BUBBLEWRAP (gated) ===
[PASS|skipped: bubblewrap not on PATH (install: apt install bubblewrap OR
[PASS] bwrap: /bin/echo runs in sandbox, stdout=plf14-bwrap-ok
[PASS] bwrap: network denied by default (curl --max-time 2 https://example
[PASS] bwrap: network allowed when policy.AllowNetwork=true (curl returns
[PASS] bwrap: deny-list rejection happens before bwrap is invoked

=== NATIVE (gated) ===
[PASS|skipped: unprivileged usersns disabled (echo 1 > /proc/sys/kernel/unp
[PASS] native: /bin/echo runs in sandbox, stdout=plf14-native-ok
[PASS] native: network denied by default
[PASS] native: deny-list rejection happens before helper is invoked
```

SUMMARY: DETECTOR=6/6 PASS; BWRAP=<n>/4 PASS; NATIVE=<n>/3 PASS

The Challenge MUST exit non-zero on any assertion failure within phases that did run. Anti-bluff smoke clean check appended to harness output. Verbatim output captured into 06_phase_1_evidence.md. Dual commit (Challenges submodule + meta-repo bump).

Subject: feat(P1-F14-T11): challenge with runtime evidence (detector/fail-closed always; bwrap + native gated).

Task 12: Close-out + push

Tick all 12 items in PROGRESS, advance PROGRESS focus to F15 candidate, run final verification (make verify-compile, anti-bluff smoke, go test -count=1 ./internal/tools/sandbox/... ./internal/commands/...), commit chore(P1-F14-T12): close out feature 14 – sandboxed shell execution, push 4 remotes non-force (origin, helixdev, vasic-digital, gitlab per programme conventions).

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types + CONST-033 (§3.3, §9), T03 detector + SelectBackend (§3.3, §5.2), T04 bubblewrap argv (§3.3, §4.3), T05 native cloneflags (§3.3, §4.4, §7), T06 manager policy enforcement (§4.2, §4.6, §5.1), T07 tool (§3.3, §3.4), T08 config loader (§3.3, §9 CONST-042), T09 slash (§3.4), T10 wiring + integration (§4.1, §5.2, §6.2), T11 Challenge three phases (§5.2, §6.3), T12 close-out (§9).
2. **TDD:** every code task starts with a failing test that exercises real code paths (CONST-033 deny-list with spawn-counter; argv builder pure-function tests; fail-closed gate against forced-empty capabilities; YAML loader against real tempdir files).
3. **Type consistency:** SandboxConfig, SandboxPolicy, SandboxCapabilities, SandboxRequest, SandboxResult, SandboxBackend, SandboxManager, SandboxedShellTool, BubblewrapBackend, NativeBackend, ConstitutionalDenyList — names match across spec §3.3 and plan T02–T08.
4. **No new external deps:** bubblewrap is invoked as a subprocess via `os/exec` (stdlib); native namespace setup uses `syscall.SysProcAttr.Cloneflags` (stdlib); YAML loading reuses `gopkg.in/yaml.v3 v3.0.1` already in `go.mod`. Confirmed via `grep -E "yaml.v3|fsnotify" helix_code/go.mod`. No `go get` needed.
5. **Anti-bluff (§5.2):** Challenge has THREE sections; Phase A always runs and asserts the verbatim fail-closed message + CONST-033 spawn-counter rejection; Phases B and C are explicitly gated with SKIP-OK: P1-F14 <missing capability> (install: <hint>). The fail-closed assertion is the single most consequential test in the feature.
6. **CONST-033 deny-list:** rejected BEFORE backend.Run is invoked; verified by `countingBackend.runs == 0` in unit + Challenge tests. List covers `systemctl {suspend|hibernate|hybrid-sleep|suspend-then-hibernate|poweroff|halt|reboot|kexec}`, bare shutdown|halt|poweroff|reboot|kexec, pm-suspend|pm-hibernate|pm-suspend-hybrid, and `loginctl {suspend|hibernate|hybrid-sleep|poweroff|reboot}`. Tokenised + word-boundary regex match catches whitespace, chained, and nested-shell variants.
7. **CONST-042:** `sandbox.yaml` written via `os.OpenFile(0_CREATE|0_WRONLY|0_EXCL, 0o600)`, parent dir at `0o700`. Loader rejects perms wider than `0600` with `ErrInsecurePerms`. Mirrors F12 wizard_writer pattern.
8. **Cross-platform:** Linux-only v1; non-Linux short-circuits to `ErrSandboxUnavailable` with platform note in §7. Build tag splits `native_backend.go (linux)` and `native_backend_other.go (!linux)`; the rest of the package builds on every GOOS so `/sandbox status` works everywhere.
9. **Existing-code constraint:** spec §3.5 documents that `internal/tools/shell/` already defines a `SandboxConfig` (and friends) but is left untouched per Q4=B. No symbol clash because the new code lives in a different package (`internal/tools/sandbox`). The bash tool stays as-is; F14 is purely additive.
10. **Branch + push:** stays on `main`, non-force to all four remotes (per CONST-043); explicit user authorization is requested at T12 before pushing.
11. **Reality check:** the existing Tool interface in `internal/tools/registry.go` (Name/Description/Schema/Execute/Category/Validate) is fully compatible with the new tool. The `SetSandboxManager` shape matches the existing F13 `SetLSPManager`, so the registry change is one method + one lazy-registration line. No registry redesign.